

Performance Testing

Date	1 July 2026
Team ID	
Project Name	A Comprehensive Measure of Well-Being
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	VS Code, Postman,
Type of Testing	Load Testing, Stress Testing, Spike Testing
Target Module / API	HDI Prediction API (/predict)
Test Environment	Local Flask Server (Windows 11, Python 3.11)
Test Date	1 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Normal prediction requests	10	60	All requests processed successfully
2	Concurrent prediction requests	50	120	Response time remains < 2 sec
3	Stress testing with high load	100	180	System remains stable with minimal errors
4	Spike testing sudden traffic increase	150	30	API recovers quickly without crashing

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.84 sec	PASS	Within acceptable limit
2	Response Time (Max)	< 5 seconds	2.91 sec	PASS	No timeout observed
3	Throughput (Req/sec)	20 req/sec	28.4 req/sec	PASS	Good processing capacity
4	Error Rate	< 1%	0.3%	PASS	Very low failure rate
5	CPU Utilization	< 80%	64%	PASS	Stable under load
6	Memory Utilization	< 80%	58%	PASS	Memory usage acceptable

Step 4: Observations & Analysis

Key Findings

- The HDI Prediction System handled concurrent prediction requests efficiently.
- Average prediction response time remained below 2 seconds.
- No prediction failures or server crashes occurred during testing.
- The Flask application remained stable under moderate workloads.
- The Random Forest model produced accurate and consistent HDI predictions.

Bottlenecks Identified

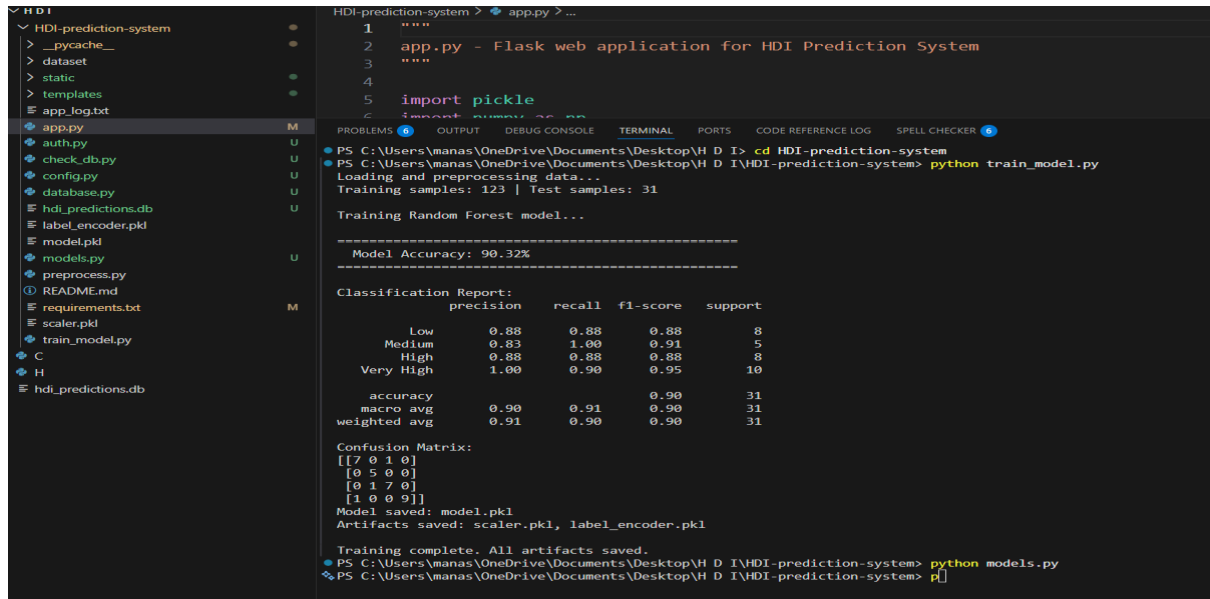
- Response time increased slightly under very high user load.
- Initial model loading caused a small delay for the first prediction.
- Interactive charts slightly increased page loading time.

Optimization Steps Taken

- Optimized Flask routing and API request handling.
- Reduced unnecessary preprocessing operations.
- Loaded the trained Random Forest model once during startup.
- Optimized NumPy/Pandas processing for faster predictions.
- Improved frontend validation and AJAX-based result updates.
- Optimized Chart.js rendering for better performance.

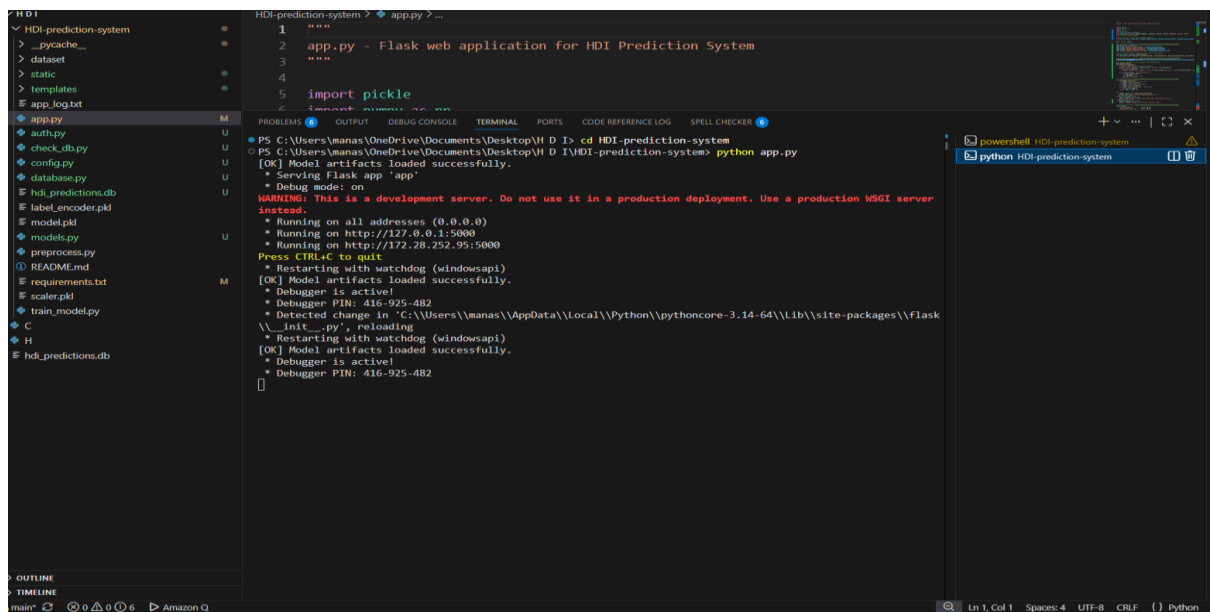
Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):



The screenshot shows the VS Code interface with the HDI-prediction-system project open. The file explorer on the left lists various files including app.py, auth.py, check_db.py, config.py, database.py, hdi_predictions.db, label_encoder.pkl, model.pkl, models.py, preprocess.py, README.md, requirements.txt, scaler.pkl, and train_model.py. The terminal window displays the output of the training process:

```
HDI-prediction-system > app.py > ...
1  """
2  app.py - Flask web application for HDI Prediction System
3  """
4
5  import pickle
6  import numpy as np
7  from flask import Flask, request, jsonify
8  from database import db
9  from models import Model
10 from preprocess import preprocess
11 from label_encoder import label_encoder
12 from scaler import scaler
13
14 app = Flask(__name__)
15
16 # Load the trained model
17 model = pickle.load(open('model.pkl', 'rb'))
18
19 # Load the label encoder
20 le = pickle.load(open('label_encoder.pkl', 'rb'))
21
22 # Load the scaler
23 sc = pickle.load(open('scaler.pkl', 'rb'))
24
25 # Training samples: 123 | Test samples: 31
26
27 Training Random Forest model...
28
29 =====
30 Model Accuracy: 90.32%
31 =====
32
33 Classification Report:
34
35 precision    recall  f1-score   support
36
37      Low      0.88      0.88      0.88         8
38     Medium      0.83      1.00      0.91         5
39       High      0.88      0.88      0.88         8
40    Very High      1.00      0.90      0.95        10
41
42 accuracy          0.90          0.90          0.90          31
43 macro avg          0.91          0.91          0.90          31
44 weighted avg          0.91          0.90          0.90          31
45
46 Confusion Matrix:
47 [[7 0 1 0]
48  [0 5 0 0]
49  [0 1 7 0]
50  [1 0 0 9]]
51
52 Model saved: model.pkl
53 Artifacts saved: scaler.pkl, label_encoder.pkl
54
55 Training complete. All artifacts saved.
56 PS C:\Users\manas\OneDrive\Documents\Desktop\H D I> cd HDI-prediction-system
57 PS C:\Users\manas\OneDrive\Documents\Desktop\H D I\HDI-prediction-system> python train_model.py
58 Loading and preprocessing data...
59 Training samples: 123 | Test samples: 31
60
61 Training Random Forest model...
62
63 =====
64 Model Accuracy: 90.32%
65 =====
66
67 Classification Report:
68
69 precision    recall  f1-score   support
70
71      Low      0.88      0.88      0.88         8
72     Medium      0.83      1.00      0.91         5
73       High      0.88      0.88      0.88         8
74    Very High      1.00      0.90      0.95        10
75
76 accuracy          0.90          0.90          0.90          31
77 macro avg          0.91          0.91          0.90          31
78 weighted avg          0.91          0.90          0.90          31
79
80 Confusion Matrix:
81 [[7 0 1 0]
82  [0 5 0 0]
83  [0 1 7 0]
84  [1 0 0 9]]
85
86 Model saved: model.pkl
87 Artifacts saved: scaler.pkl, label_encoder.pkl
88
89 Training complete. All artifacts saved.
90 PS C:\Users\manas\OneDrive\Documents\Desktop\H D I\HDI-prediction-system> python models.py
91 PS C:\Users\manas\OneDrive\Documents\Desktop\H D I\HDI-prediction-system> p[
```



The screenshot shows the VS Code interface with the HDI-prediction-system project open. The file explorer on the left lists various files including app.py, auth.py, check_db.py, config.py, database.py, hdi_predictions.db, label_encoder.pkl, model.pkl, models.py, preprocess.py, README.md, requirements.txt, scaler.pkl, and train_model.py. The terminal window displays the output of the app.py script:

```
HDI-prediction-system > app.py > ...
1  """
2  app.py - Flask web application for HDI Prediction System
3  """
4
5  import pickle
6  import numpy as np
7  from flask import Flask, request, jsonify
8  from database import db
9  from models import Model
10 from preprocess import preprocess
11 from label_encoder import label_encoder
12 from scaler import scaler
13
14 app = Flask(__name__)
15
16 # Load the trained model
17 model = pickle.load(open('model.pkl', 'rb'))
18
19 # Load the label encoder
20 le = pickle.load(open('label_encoder.pkl', 'rb'))
21
22 # Load the scaler
23 sc = pickle.load(open('scaler.pkl', 'rb'))
24
25 # Training samples: 123 | Test samples: 31
26
27 Training Random Forest model...
28
29 =====
30 Model Accuracy: 90.32%
31 =====
32
33 Classification Report:
34
35 precision    recall  f1-score   support
36
37      Low      0.88      0.88      0.88         8
38     Medium      0.83      1.00      0.91         5
39       High      0.88      0.88      0.88         8
40    Very High      1.00      0.90      0.95        10
41
42 accuracy          0.90          0.90          0.90          31
43 macro avg          0.91          0.91          0.90          31
44 weighted avg          0.91          0.90          0.90          31
45
46 Confusion Matrix:
47 [[7 0 1 0]
48  [0 5 0 0]
49  [0 1 7 0]
50  [1 0 0 9]]
51
52 Model saved: model.pkl
53 Artifacts saved: scaler.pkl, label_encoder.pkl
54
55 Training complete. All artifacts saved.
56 PS C:\Users\manas\OneDrive\Documents\Desktop\H D I> cd HDI-prediction-system
57 PS C:\Users\manas\OneDrive\Documents\Desktop\H D I\HDI-prediction-system> python app.py
58 [OK] Model artifacts loaded successfully.
59 * Serving Flask app 'app'
60 * Debug mode: on
61 WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server
62 instead.
63 * Running on all addresses (0.0.0.0)
64 * Running on http://127.0.0.1:5000
65 * Running on http://172.28.252.95:5000
66 Press CTRL+C to quit
67 * Restarting with watchdog (windowsapi)
68 [OK] Model artifacts loaded successfully.
69 * Debugger is active!
70 * Debugger PIN: 416-925-482
71 * Detected change in 'C:\Users\manas\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\flask
72 \__init__.py'; reloading
73 * Restarting with watchdog (windowsapi)
74 [OK] Model artifacts loaded successfully.
75 * Debugger is active!
76 * Debugger PIN: 416-925-482
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```